

Progetto Machine and Reinforcement Learning

DI MIRKO DAMIANO & LUCA CALLEGARI

Matricole: 0378007 - 0375344

Università degli studi Roma “Tor Vergata”
Corso di Laurea Magistrale *Ingegneria dell’Automazione*

Indice

1 Task 1 - Multi-arm Bandits	2
1.1 Caso stazionario	3
1.2 Caso non stazionario	3
1.3 Caso UCB	4
1.4 Caso con preferenze	5
2 Task 2 - Gambler	7
3 Task 3 - Racetrack	10
4 Progetto finale - Trading online	12
4.1 Ambiente	12
4.2 Agente SARSA	13
4.3 Agente Q-LEARNING	14
4.4 Agente DOUBLE Q-LEARNING	14
4.5 S&P 500	15

1 Task 1 - Multi-arm Bandits

Il primo task consiste nel modellare il famoso gioco **Rock, Paper, Scissors, Lizard, Spock** mostrato nella puntata 5×17 della serie tv *The Big Bang Theory*:

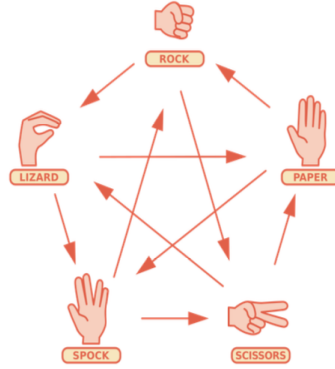


Figura 1.

Lo scopo di tale task consiste nell'addestrare un bot a vincere contro un avversario che può scegliere l'azione in modo casuale, deterministico o probabilistico. Per di più è stato lasciato all'utente la possibilità di decidere, tra le seguenti, quale tipologia di algoritmo utilizzare per l'addestramento:

1. Problema stazionario, in cui i reward vengono pescati da una distribuzione fissa nel tempo;
2. Problema non stazionario;
3. Upper Confidence Bound, in cui si privilegiano le azioni sub-ottime;
4. Preferenze, in cui si associa ad ogni azione una preferenza $H_t(a)$

Le mosse vengono indicizzate in un vettore $\mathcal{A}$ di 5 elementi con le seguenti codifiche:

$$\begin{cases} \text{paper} = 1 \\ \text{scissors} = 2 \\ \text{rock} = 3 \\ \text{lizard} = 4 \\ \text{spock} = 5 \end{cases}$$

affinchè le azioni scelte dall'avversario e dal bot vengano confrontate fra loro per assegnare al bot un relativo reward; in particolare si ha un reward positivo +1 in caso di vittoria, un reward negativo -1 in caso di sconfitta ed un reward nullo in caso di pareggio.

Osservazione 1. Per promuovere l'esplorazione all'inizio dell'episodio, quando non si hanno informazioni sull'ambiente, è stata utilizzata la tecnica di **inizializzazione ottimistica**. Per di più in tale task l'ambiente di lavoro è di tipo **non associativo**

1.1 Caso stazionario

Se l'utente ha selezionato la prima opzione, il bot sceglie l'azione in maniera ε -greedy:

$$A_t \in \begin{cases} \arg \max_a Q_t(a), & \text{con probabilità } 1 - \varepsilon \\ \mathcal{A}, & \text{con probabilità } \varepsilon \end{cases}$$

la quale viene tracciata incrementando il contatore $N(A_t)$; ciò è necessario per poter aggiornare correttamente l'elemento della funzione qualità, relativo all'azione A_t , nel seguente modo:

$$Q(A_t) = Q(A_t) + \frac{1}{N(A_t)}(R - Q(A_t))$$

A titolo di esempio consideriamo il caso in cui l'avversario basa le proprie scelte secondo il vettore di probabilità:

$$\mathbb{P}_{\text{scelta}} = \left[\frac{1}{5}, \frac{3}{5}, 0, \frac{1}{5}, 0 \right]$$

per cui il risultato atteso sarà sicuramente dato dalla tendenza del bot a scegliere rock oppure spock.

Osservazione 2. Per i risultati successivi sono stati considerati i seguenti parametri: $\alpha = 0.01$, $\varepsilon = 0.2$, $c = 0.5$

Possiamo appunto notare dalla seguente figura, il risultato previsto poc'anzi:

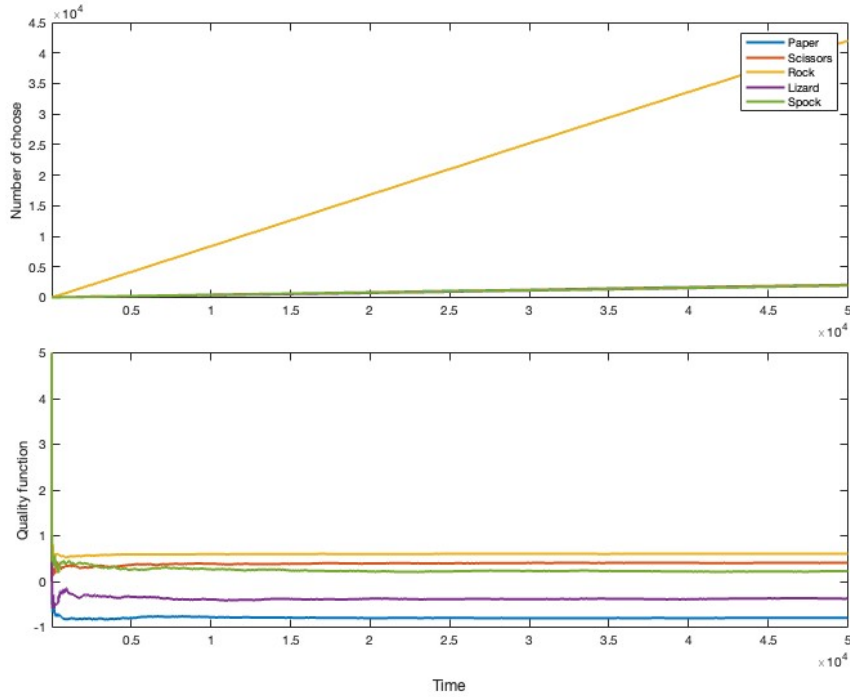


Figura 2.

in cui facciamo notare anche come la bontà delle due azioni ottime si rispecchia nell'andamento della funzione qualità.

1.2 Caso non stazionario

Per tale caso il modo in cui l'azione viene scelta dal bot si basa sempre sulla politica ε -greedy per cui l'unica differenza con il precedente caso consiste nel considerare un passo d'aggiornamento costante, con parametro $\alpha \in (0, 1]$:

$$Q_{k+1}(a) = Q_k(a) + \alpha (R_k - Q_k(a))$$

In particolare esso dovrebbe essere normalizzato affinché siano rispettate le condizioni di convergenza stocastica:

$$\sum_{k=1}^{\infty} \alpha_k(a) = \infty, \quad \sum_{k=1}^{\infty} \alpha_k(a)^2 < \infty$$

ma in tal caso l'aggiornamento della funzione qualità peggiorerebbe per $t \rightarrow \infty$, in quanto non si cambierebbero le scelte in base all'ambiente.

Mostriamo pertanto il risultato ottenuto considerando lo stesso esempio illustrato precedentemente:

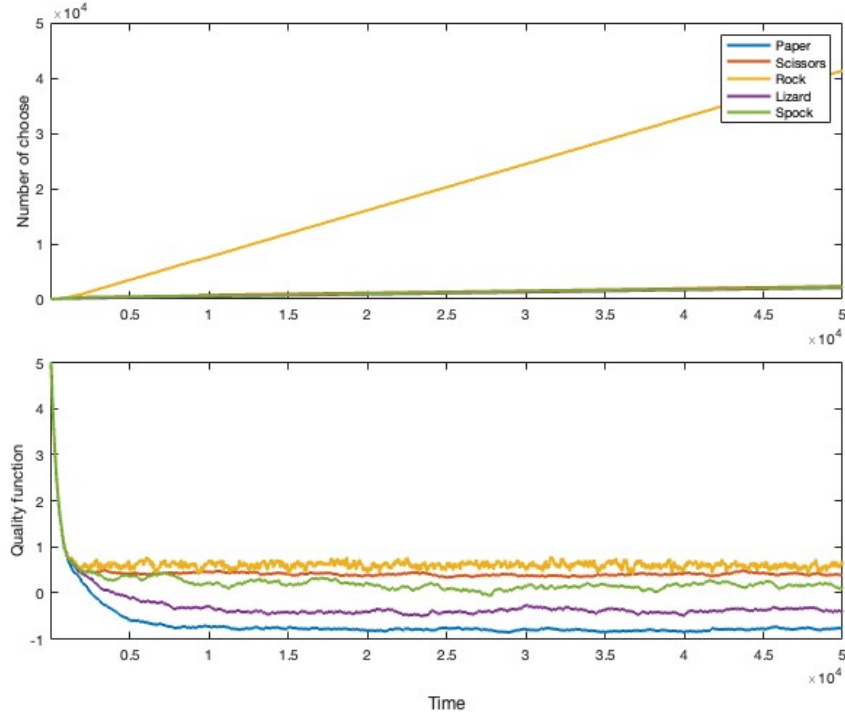


Figura 3.

ove facciamo notare che l'andamento delle funzioni qualità sia oscillante, poichè esse vengono continuamente aggiornate.

1.3 Caso UCB

Se invece l'utente ha scelto la terza opzione, il bot sceglie un'azione sulla base della stima della funzione qualità considerando la seguente formula:

$$A_t = \arg \max_a \left\{ Q_t(a) + c \sqrt{\frac{\ln(t)}{N_t(a)}} \right\}, \quad c > 0$$

ove il secondo addendo è un'incertezza della funzione qualità, perchè nel caso ϵ -greedy un'azione sub-ottima e una pessima hanno eguale probabilità di essere scelte.

Osservazione 3. La legge di aggiornamento della funzione qualità rimane invariata rispetto al precedente caso

Nella seguente figura il risultato ottenuto:

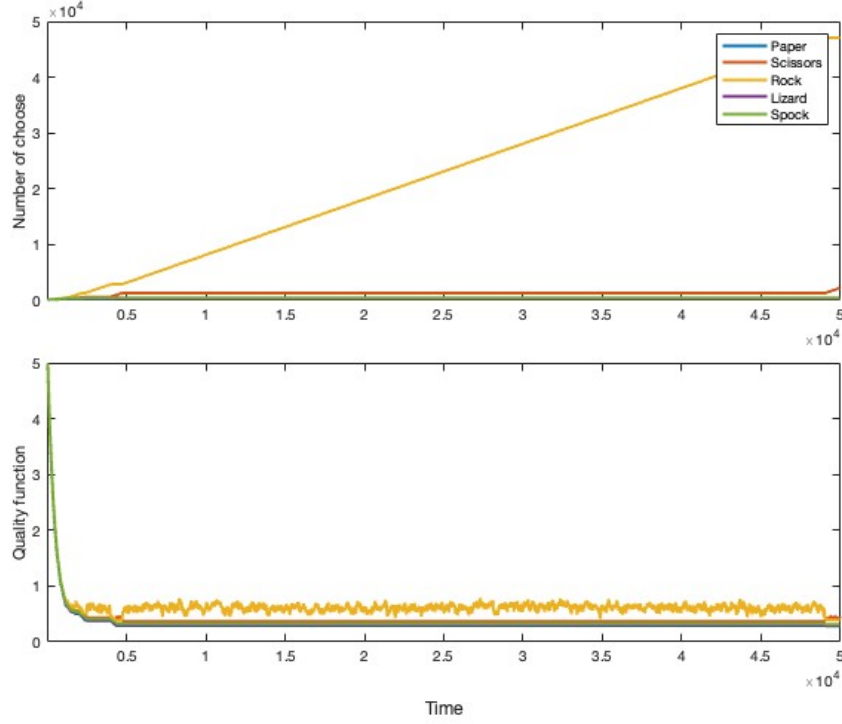


Figura 4.

1.4 Caso con preferenze

Invece che avere una stima della funzione qualità e scegliere le azioni secondo una politica ϵ -greedy o *UCB*, si associa ad ogni azione una preferenza $H_t(a)$.

Se la preferenza è alta si sceglie l'azione con probabilità più elevata e viceversa; da un punto di vista matematico si utilizza una distribuzione denominata *soft max*:

$$\mathbb{P}[A_t = a] = \frac{e^{H_t(a)}}{\sum_{b=1}^n e^{H_t(b)}} = \pi_t(a)$$

Dopo aver selezionato l'azione A_t ed aver ricevuto il reward R_t , supponiamo che inizialmente le preferenze siano tutte eguali, aggiornate mediante la seguente legge:

$$\begin{cases} H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)) \\ H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a), \quad \forall a \neq A_t \end{cases}$$

ove $\bar{R}_t$ è il reward medio ottenuto fino all'istante di tempo corrente; tuttavia, la differenza $(R_t - \bar{R}_t)$ può essere definita come errore di misura.

Se il reward ottenuto è maggiore della media allora la probabilità di scegliere quell'azione aumenta; viceversa se il reward ottenuto è minore della media allora la probabilità diminuisce.

Anche in tal caso l'avversario basa le proprie scelte sul vettore di probabilità:

$$\mathbb{P}_{\text{scelta}} = \left[\frac{1}{5}, \frac{3}{5}, 0, \frac{1}{5}, 0 \right]$$

con seguenti risultati:

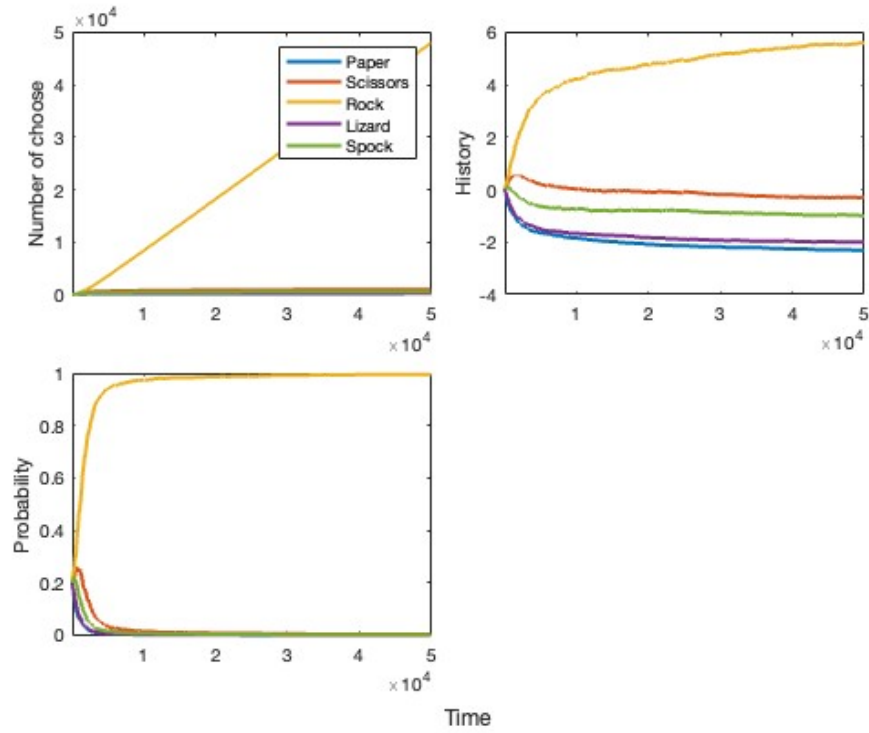


Figura 5.

in cui notiamo che l'azione ottima, ossia scegliere rock, tende ad essere scelta con probabilità 1 rispetto alle altre azioni.

2 Task 2 - Gambler

Il Model gambler's problem descrive un giocatore d'azzardo che scommette somme di denaro su una serie di lanci di moneta equa. Se esce testa, il giocatore vince una quantità di denaro pari alla puntata; se esce croce, la perde.

L'obiettivo consiste nel raggiungere una determinata somma pari a 100\$, pertanto il problema termina al raggiungimento di tale obiettivo o nel momento in cui il giocatore non ha saldo disponibile.

Il codice implementa la struttura del modello attraverso i seguenti elementi:

- lo **stato** s rappresenta il saldo attuale del giocatore, includendo tutti i valori da 0 a 100, ove lo stato 0 di fallimento e lo stato 100 di vittoria sono definiti come stati terminali;
- l'**azione** a rappresenta la puntata scommessa dal giocatore

Al variare di questi due parametri viene inizialmente costruita la matrice di transizione degli stati $P(s, s', a)$, che indica la probabilità di passare dallo stato s allo stato successivo s' scegliendo l'azione a .

Il primo caso che viene gestito riguarda gli stati terminali, in quanto se il giocatore possiede 0\$ o 100\$ rimane in quello stato con probabilità 1; se invece il giocatore ha possibilità di puntare allora la puntata massima sarà data da:

$$a_true = \min(a, s)$$

poichè la puntata deve essere minore o uguale del saldo attuale.

Quest'ultima permette di aggiornare il saldo attuale nel seguente modo:

- in caso di vincita: $sp_win = s + a_true$, fino ad un massimo di 100\$;
- in caso di perdita: $sp_lose = s - a_true$, fino ad un minimo di 0\$

e conseguentemente la matrice di transizione $P(s, s', a)$, in entrambi i casi, viene aggiornata con probabilità $\frac{1}{2}$.

Ogni componente della matrice dei reward $R(s, a)$ viene invece calcolata come segue:

$$r(s, a) = \sum_{r \in R} r \sum_{s' \in S} p(s', r | s, a)$$

e sostanzialmente, il giocatore riceve un premio di +1 se ha raggiunto l'obiettivo e una penalità di -1 se ha fallito; in tutti gli altri casi il reward è nullo. Pertanto il codice implementa tale formula:

$$R(s, a) = P(s, S, a) * 1 + P(s, 1, a) * (-1)$$

in cui S e 1 rappresentano gli stati terminali.

Osservazione 4. La probabilità di ottenere un qualsiasi reward e finire in un qualsiasi stato successivo, a partire da una coppia stato-azione, è pari ad uno:

$$\sum_{s' \in S} \sum_{r \in R} p(s', r | s, a) = 1, \quad \forall a \in \mathcal{A}(s), \forall s \in S$$

poichè l'insieme dei reward e degli stati successivi è finito

Mostriamo infine il grafico rappresentante l'andamento delle policy ottime **pi** e **pistar**, calcolate rispettivamente con gli algoritmi di *value iteration* e *policy iteration*:

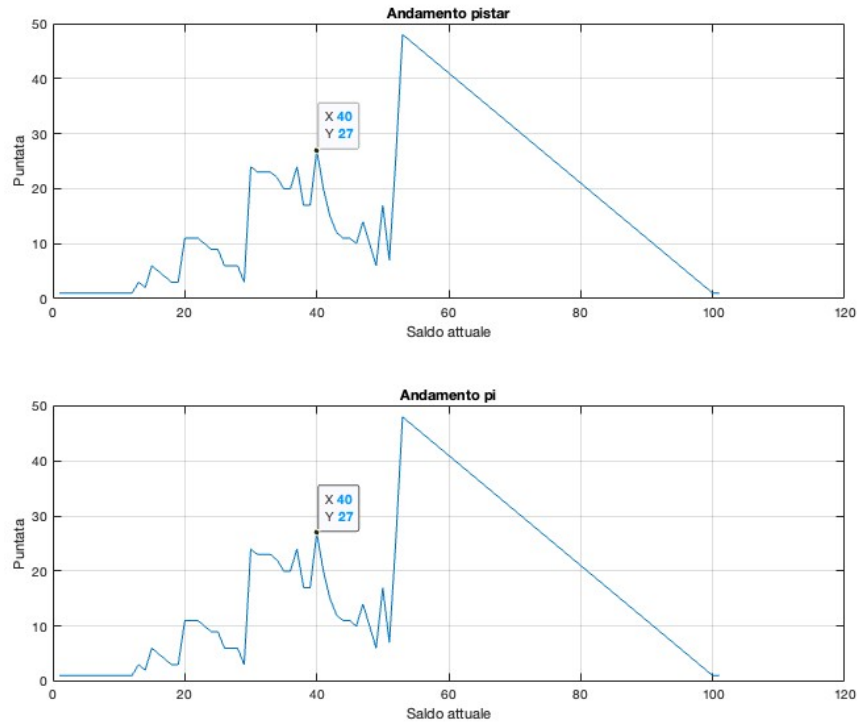


Figura 6.

in cui le policy, dipendenti dallo stato corrente, permettono di ottenere l'azione ottima in feedback dallo stato ed in cui possiamo notare l'uguaglianza di essi, poichè entrambi convergono alla soluzione dell'equazione di ottimalità Bellman.

Un secondo grafico importante riguarda il numero di iterazioni necessarie alla convergenza delle due policy ottime:

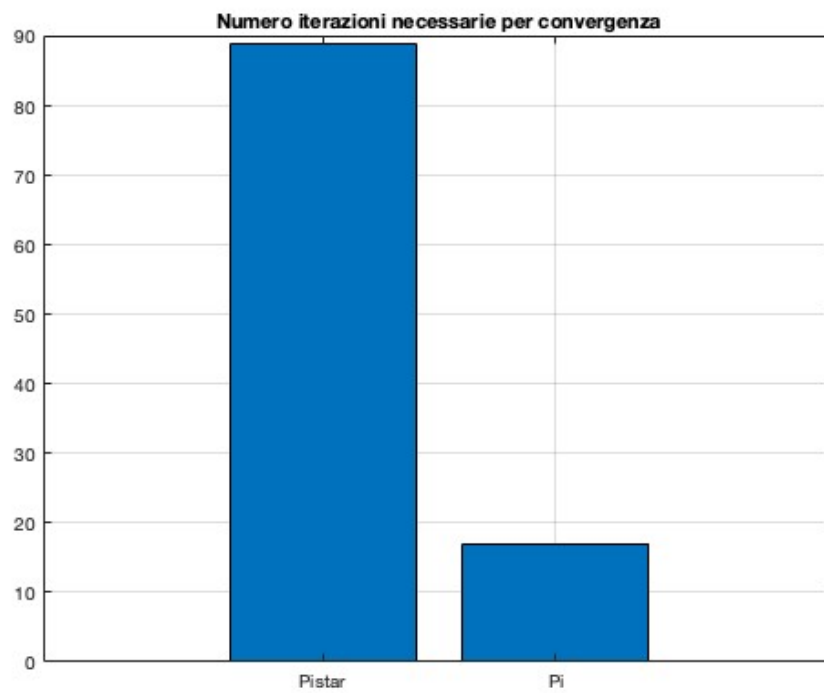


Figura 7.

ove si nota come l'algoritmo di *value iteration* richieda più iterazioni del *policy iteration*, in quanto la funzione valore viene continuamente aggiornata finché il processo iterativo non scende sotto una determinata tolleranza.

3 Task 3 - Racetrack

L'obiettivo primario del Racetrack consiste nel guidare un veicolo lungo una pista bidimensionale, partendo da una linea di avvio e raggiungendo il traguardo nel minor numero di passi temporali possibile, evitando collisioni con i bordi del tracciato o con eventuali ostacoli.

L'ambiente è rappresentato da una griglia discreta di celle, `trackMap`, e lo stato del sistema è definito da una quadrupla di variabili intere:

`currentState = [row, column, velX, velY]`

ove `[row, column]` indicano le coordinate spaziali del veicolo sulla griglia e `[velX, velY]` rappresentano le componenti della velocità, rispettivamente lungo l'asse x e lungo l'asse y ; quest'ultime sono vincolate nell'intervallo:

$$-5 < v_x, v_y < 5$$

con la condizione aggiuntiva che i due valori non siano entrambi nulli, eccetto che sulla linea di partenza così da evitare lo stallo permanente dell'agente, problema noto come **The Lazy Agent**, in quanto l'agente tende a capire che l'unico modo di vincere è non giocare.

Inoltre l'agente interagisce con l'ambiente attraverso un set di azioni discrete, codificate come segue:

$$\begin{cases} -1 = \text{accelerazione negativa} \\ 0 = \text{accelerazione nulla} \\ 1 = \text{accelerazione positiva} \end{cases}$$

applicate ad ambedue gli assi x, y . Ai fini poi della programmazione ogni componente di velocità e accelerazione è stata mappata in un vettore di indici compatibile con la logica di Matlab.

Ad ogni passo temporale la posizione della macchina viene aggiornata in base alla velocità corrente seguendo la cinematica discreta:

$$\begin{cases} \text{new_row} = \text{current_row} + \text{new_velY} \\ \text{new_column} = \text{current_column} + \text{new_velX} \end{cases}$$

e conseguentemente vengono effettuati tre tipi di controlli:

1. **controllo sulla vittoria:** si verifica se la macchina ha superato l'ultima colonna della griglia, ossia ha raggiunto la Finish Line;
2. **controllo sui confini:** si verifica se la traiettoria calcolata interseca il confine della pista;
3. **controllo sugli ostacoli:** si verifica se c'è stato un evento di collisione

In particolare nel secondo e nel terzo caso, la dinamica prevede il reset dell'auto mediante la variabile booleana `crushed`, con conseguente ricollocamento della stessa in una posizione casuale della linea di partenza.

Per quanto riguarda la distribuzione delle ricompense all'agente, è stata implementata una strategia che prevede di assegnare `reward = -1000` ogni qualvolta la macchina collida, varchi i confini del tracciato o persista in una cella della griglia con velocità nulla, mentre `reward = +1000` in caso di vittoria. Altrimenti la ricompensa assegnata all'agente è costante e pari a `-1`.

Osservazione 5. Poichè il contesto in cui lavora l'agente è di tipo episodico, sono stati implementati due tipologie di algoritmi: Monte Carlo (*every visit*) e SARSA.

Mostriamo infine la simulazione in cui l'agente impara a guidare la macchina fino a raggiungere l'obiettivo:

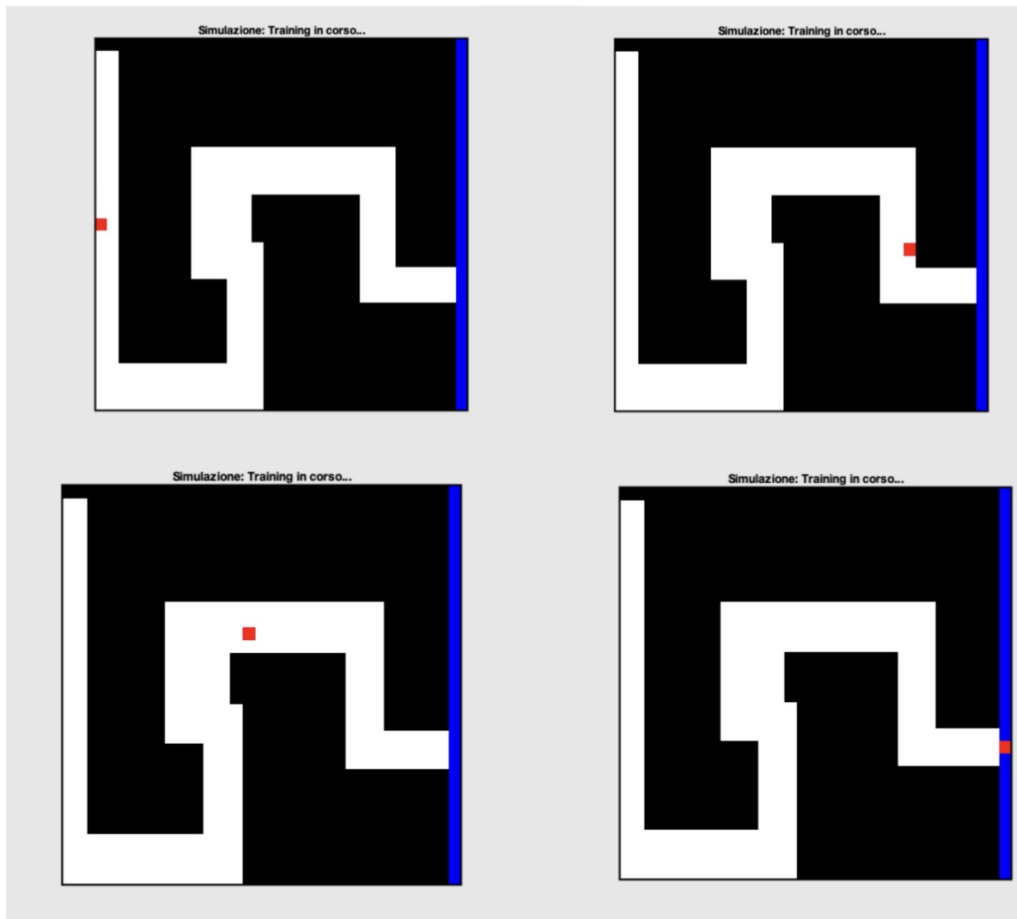


Figura 8.

4 Progetto finale - Trading online

Il progetto conclusivo di Machine and Reinforcement Learning consiste nell'addestrare un agente virtuale a lavorare in un ambiente di trading online; quest'ultimo viene pertanto simulato mediante la libreria open-source **yfinance**: essa permette di scaricare dati finanziari da *Yahoo Finance* su azioni, criptovalute e altro, per cui i tickers scelti, nell'intervallo di tempo 2020 – 2024, sono i seguenti:

```
tickers = ["BTC-USD", "ETH-USD", "GLD", "AAPL", "MSFT"]
```

Osservazione 6. *I dati storici vengono salvati all'interno di un file CSV, denominato market_prices_rl.csv*

4.1 Ambiente

Nello specifico l'agente percepisce l'ambiente attraverso un vettore di stato definito come segue:

```
state = np.concatenate([
    [self.cash_disponibile],
    [valore_portafoglio],
    prezzi_attuali,
    self.quantita_possedute
])
```

in cui il primo campo rappresenta la liquidità disponibile, a partire da un capitale iniziale di 10.000\$, il secondo campo viene aggiornato sommando alla liquidità il controvalore degli asset:

```
valore_portafoglio = cash_disponibile + sum(quantita_possedute * prezzi_attuali)
```

mentre gli ultimi due campi indicano il vettore dei prezzi attuali e il vettore delle quantità possedute, per ogni asset rispettivamente.

Prima di presentare la logica delle operazioni che l'agente effettua sul mercato finanziario, risulta necessario definire le seguenti variabili di controllo:

- **TARGET_EXPOSURE**: l'agente evita che un ticker occupi più del 15% del valore totale del portafoglio, garantendo la diversificazione;
- **MAX_SELL_PERCENT**: è la massima percentuale che l'agente decide di liquidare per ogni asset, in caso di vendita;
- **ESPOSIZIONE**: è il rapporto tra il valore dell'asset posseduto e il valore totale del portafoglio

$$\text{esposizione} = \frac{\text{quantità} \times \text{prezzo asset}}{\text{valore portafoglio}}$$

Ogni giornata di trading prevede pertanto che l'agente scelga 3 azioni possibili:

- **BUY**: l'agente non investe tutto il capitale in un colpo solo, ma la quota investita prevede che il cash disponibile sia moltiplicato per la percentuale di asset acquistabile:

```
perc_asset_buy = max(0.05, TARGET_EXPOSURE - esposizione)
```

- **SELL:** l'agente non liquida l'intera quantità di asset posseduta, ma ne vende una data percentuale:

$$\text{perc_asset_sell} = \text{MAX_SELL_PERCENT} * \text{esposizione}$$

- **HOLD:** l'agente non effettua nessuna delle precedenti operazioni sul mercato

Inoltre l'agente basa il proprio apprendimento sul segnale di reward che riceve dopo ogni azione:

$$R_{t+1} = \frac{\text{portafoglio}_{t+1} - \text{portafoglio}_t}{\text{portafoglio}_t}$$

4.2 Agente SARSA

Un primo tipo di agente si basa sull'algoritmo on-policy *SARSA*, aggiornando la funzione valore $Q(s, a)$ mediante la policy seguita dall'agente stesso e includendo l'azione successiva a' scelta; ciò lo rende più prudente e consapevole del rischio presente durante l'esplorazione.

Poichè lo spazio degli stati è continuo l'agente utilizza il **Tile Coding**¹, tramite una **IHT**(Index Hash Table), per mappare lo stato in un insieme di 32 indici discreti.

In tal modo stati simili attiveranno indici comuni, consentendo all'agente di capire cosa fare in situazioni di mercato mai viste prima, ma simili a quelle passate.

Per ogni asset, l'aggiornamento della funzione qualità segue la regola:

$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

in cui sia l'azione corrente A che l'azione successiva A' vengono scelte in modo ε -greedy. Tuttavia la funzione qualità viene aggiornata, alla fine di ogni episodio, mediante un approccio di tipo **batch** in quanto l'interazione diretta con l'ambiente è rischiosa.

Mostriamo pertanto l'andamento del portafoglio e dell'errore relativo della funzione valore:

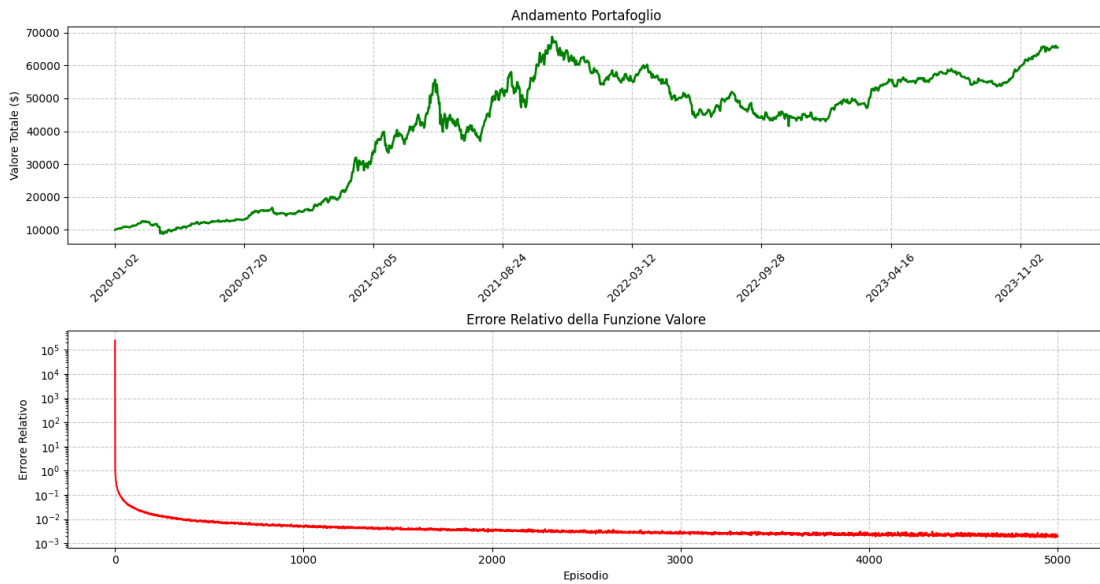


Figura 9.

1. Il codice sorgente del Tile Coding è stato acquisito dal sito <http://incompleteideas.net/tiles/tiles3.html>, by Rich Sutton

4.3 Agente Q-LEARNING

Nel caso in cui l'agente si basi sull'algoritmo off-policy *Q-LEARNING*, solamente l'azione A viene scelta sempre in maniera ε -greedy e, a differenza del precedente algoritmo, si stima direttamente la funzione qualità ottima. Infatti le equazioni di ottimalità di Bellman, che non dipendono dalla policy, vengono utilizzate per effettuare l'aggiornamento della funzione qualità come segue:

$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma \max_a Q(S', a) - Q(S, A))$$

Osservazione 7. Anche in tal caso il *Tile Coding* è stato utilizzato per discretizzare lo stato e la politica si basa su un approccio di tipo batch

Nel seguente grafico possiamo infatti notare come il guadagno effettivo sia inferiore rispetto a quello ottenuto con l'algoritmo *SARSA*, in quanto l'agente tende a rischiare molto di più perchè considera solamente l'azione che massimizza la funzione qualità:

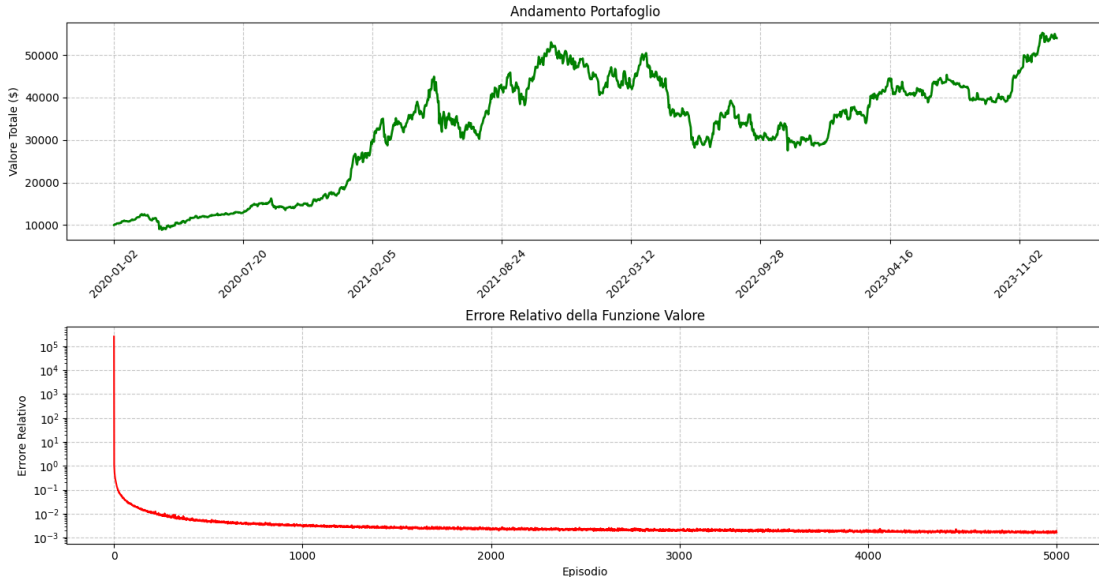


Figura 10.

4.4 Agente DOUBLE Q-LEARNING

Nell'aggiornamento della funzione qualità il termine che la massimizza viene utilizzato sia per scegliere l'azione da considerare nello stato successivo, sia per valutarla.

Questa situazione potrebbe dare origine ad un problema noto come **sovrastima ottimistica** o **positive bias**, producendo una funzione qualità che sopravvaluta il reale valore delle azioni, con conseguente policy sub-ottimale o instabile.

Il *DOUBLE Q-LEARNING* è una soluzione che prevede l'utilizzo di due stime distinte della funzione qualità Q_1 e Q_2 nel seguente modo:

- Q_1 viene utilizzata per scegliere l'azione ottima:

$$A_t^* = \arg \max_{a \in \mathcal{A}} Q_1(S_t, a)$$

- Q_2 viene utilizzata per stimare il valore dell'azione ottima:

$$Q_2(S_t, A_t^*) = Q_2 \left(S_t, \arg \max_{a \in \mathcal{A}} Q_1(S_t, a) \right)$$

Osservazione 8. Anche per tale agente il *Tile Coding* è stato utilizzato per discretizzare lo stato e la politica si basa su un approccio di tipo batch

Pertanto l'aggiornamento delle funzioni qualità viene eseguito come segue:

$$\begin{cases} Q_1(S, A) \leftarrow Q_1(S, A) + \alpha (R + \gamma Q_2(S', \arg \max_a Q_1(S', a)) - Q_1(S, A)), & \text{con probabilità } 0.5 \\ Q_2(S, A) \leftarrow Q_2(S, A) + \alpha (R + \gamma Q_1(S', \arg \max_a Q_2(S', a)) - Q_2(S, A)), & \text{altrimenti} \end{cases}$$

Mostriamo infine il relativo grafico:

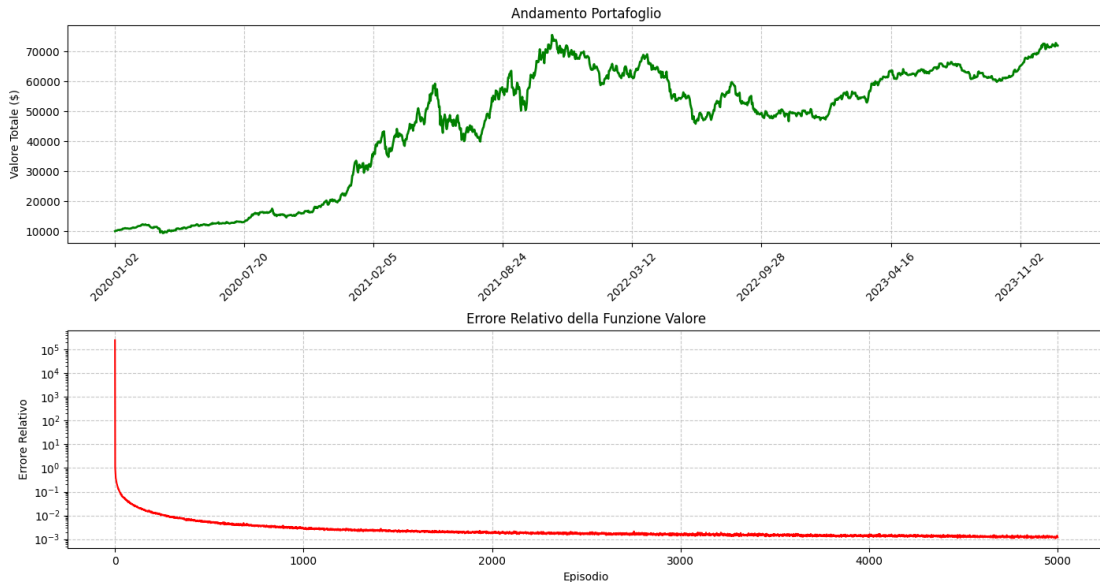


Figura 11.

4.5 S&P 500

L'*S&P 500* è uno dei più importanti indici azionari al mondo; esso traccia l'andamento di mercato delle 500 aziende statunitensi a maggior capitalizzazione quotate sulle borse di New York. A differenza di un singolo asset, come *AAPL*, l'*S&P 500* è considerato il benchmark universale per misurare la performance di qualsiasi portafoglio.

In tale contesto confrontare i risultati con l'*S&P 500* è fondamentale per:

- verificare se l'agente creato può generare profitti superiori a tale andamento, poichè un investitore avrebbe ottenuto un risultato migliore semplicemente acquistando l'indice e non lavorare direttamente sul mercato azionario;
- garantire ottime performance perchè battere l'*S&P 500* è l'obiettivo di ogni sviluppatore di trading bot

Inoltre poichè l'agente opera su asset 24/7, come cryptovalute, i dati dell'*S&P 500* sono stati allineati con il file CSV su cui lavora l'agente; ciò garantisce che l'*S&P 500* abbia a disposizione i dati delle azioni anche quando teoricamente la borsa sarebbe chiusa.

Osservazione 9. L'*S&P 500* è una strategia passiva a differenza dell'agente dinamico che effettua operazioni *BUY-SELL* sul mercato azionario

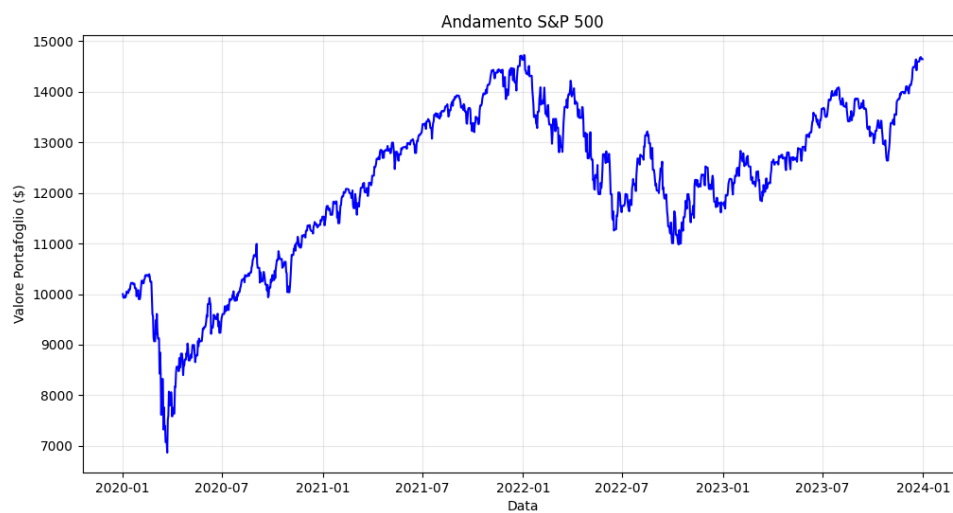


Figura 12.